

EXAM POV NOTES (Q&A FORMAT)**10 Marks**

Q1: Define a Graph Database (GDB) and explain how its fundamental structures (nodes, edges, and properties) represent data compared to the tabular structure of a Relational Database Management System (RDBMS), detailing the equivalents for tables, rows, columns, constraints, and joins.

A Graph Database is a specialized NoSQL database that models data using graph theory concepts, focusing intensely on relationships rather than rigid table structures.

- **Definition of GDB:** A graph database (GDB) is a database that uses graph structures (nodes, edges, and properties) for storing data, and they are commonly referred to as a sub-type of NoSQL databases.
- **Fundamental Structures:** GDBs use nodes (entities), edges (relationships), and properties (key-value pairs) instead of tables or documents to represent and store data.
- **Role of Edges:** The edges explicitly represent relationships between nodes, which is the key factor that allows for easy data retrieval, often achievable in a single operation.
- **RDBMS Table Equivalent:** The high-level structure of Tables in RDBMS is conceptually represented by Graphs in a GDB.
- **RDBMS Row Equivalent:** Individual Rows in an RDBMS are equivalent to Nodes (entities) in a GDB.
- **RDBMS Column/Data Equivalent:** The Columns and Data stored in RDBMS are analogous to Properties and their values stored on nodes and relationships in the GDB.
- **RDBMS Constraint Equivalent:** Structural Constraints used in RDBMS are represented by Relationships (edges) in a GDB.
- **RDBMS Join Equivalent:** The complex operation of Joins used in RDBMS to link data is replaced by Traversal operations in a GDB.
- **Schema Flexibility (Neo4j):** Neo4j, which follows the native property graph model, offers a flexible data model where there is no need to follow a fixed schema, allowing properties to be added or removed as required, although it still provides schema constraints.

Q2: Discuss the challenges associated with scaling Graph Databases, particularly concerning sharding, and describe the three general ways to scale them, including techniques to improve read scaling.

Scaling GDBs is challenging because their relationship-oriented nature conflicts with typical sharding techniques designed for aggregate data.

- **Sharding Difficulty:** Sharding—where data is split and distributed across different servers—is a common NoSQL technique but is difficult in graph databases.
- **Reason for Difficulty:** This difficulty arises because GDBs are relationship-oriented, meaning any given node can potentially be related to any other node, unlike aggregate-oriented databases.
- **Performance Impact of Distribution:** If related nodes are traversed when they are stored on different machines, the overall performance of the graph traversal degrades significantly.
- **Optimal Node Placement:** To maintain performance, storing related nodes on the same server is generally required to optimize graph traversal.
- **Scaling Method 1: RAM Expansion:** One way to scale is by adding substantial RAM to the server so that the working set of nodes and relationships is held entirely in memory.
- **Limitation of RAM Scaling:** This technique is only effective if the specific dataset being worked with will fit into a realistic amount of RAM.

- Scaling Method 2: Improved Read Scaling: Read scaling can be improved by adding more slaves that are configured with read-only access to the data.
- Write Traffic in Read Scaling: When employing the read scaling method using slaves, all write operations are typically directed to the master server.
- Scaling Method 3: Application-Side Sharding: When the large size of the dataset renders replication impractical, data sharding can be performed from the application side.
- Requirement for App Sharding: Application-side sharding requires the application to use domain-specific knowledge to appropriately split and manage the data distribution.

5 Marks

Q5: When is a Graph Database needed? Provide at least two reasons, specifically addressing complex queries in relational databases and the importance of relationships between data elements.

Deception: GDBs are needed when connectivity and complex inter-data relationships are the primary focus of queries and data modeling.

- Solving Many-to-Many Relationships: GDBs are needed to effectively solve complex Many-To-Many relationship problems, such as tracking "friends of friends" in a social graph.
- Relational Query Complexity: They are required when the necessary query in a traditional relational database becomes very complex, often involving numerous joins.
- Focus on Relationships: GDBs are necessary when the relationships between data elements are more important than the individual properties of the data elements themselves.
- Low Latency with Large Relationships: They address the issue of achieving low latency when dealing with large-scale datasets containing numerous relationships, which would otherwise result in huge datasets and increased query complexity and time in RDBMS.
- Link-Rich Domains: Any link-rich domain, such as modeling social networks or tracking employee knowledge and project collaboration, is well suited for deploying graph databases.

Q6: Explain the concept of availability in Neo4j, describing how high availability is achieved through replicated slaves and how writes are handled in a master/slave setup.

Deception: Neo4j ensures high availability (HA) by replicating data to slaves that can handle reads and managed writes.

- High Availability (HA) Mechanism: Neo4j, as of version 1.8, achieves high availability by providing replicated slaves.
- Slave Read Functionality: In a cluster setup, slaves are always available to service read operations.
- Slave Write Capability: These replicated slaves are also capable of handling write operations.
- Write Synchronization to Master: When a write operation is directed to a slave, that slave must synchronize the write to the current master server.
- Commit Process: The write is committed first at the master node and then committed at the slave node that initiated the write; other slaves will eventually receive the update.

Q7: Detail the mechanism by which consistency is ensured in graph databases, including the role of transactions and the rule regarding deleting nodes.

Consistency in GDBs is maintained via ACID-compliant transactions and strict rules against dangling relationships.

- **Primary Mechanism:** Graph databases ensure consistency primarily through the implementation of transactions.
- **ACID Compliance:** Neo4j is specifically fully ACID-compliant (Atomicity, Consistency, Isolation, Durability), ensuring that data remains consistent within a single server.
- **Dangling Relationship Prevention (Existence):** Consistency rules prohibit "dangling relationships"; thus, the start node and the end node of any relationship must always exist.
- **Dangling Relationship Prevention (Deletion):** Nodes can only be deleted if they do not have any relationships attached to them, further enforcing data integrity.
- **Cluster Consistency (Eventually Consistent):** While a write to the master is eventually synchronized to the slaves in a cluster, other slaves are not synchronized immediately and must wait for the data to propagate from the master.

Q8: Describe the query features supported by Graph Databases, including the function of query languages like Gremlin and Cypher, and the use of indexing.

GDBs leverage specialized traversal languages and indexing to efficiently navigate their relationship-centric structure.

- **Query Language Support:** Graph databases are supported by dedicated query languages, notably Gremlin and Neo4j's Cypher.
- **Gremlin Function:** Gremlin is a domain-specific language whose purpose is specifically defined for traversing graphs.
- **Cypher Function:** Neo4j uses Cypher, a powerful, declarative query language that employs ASCII-art to depict graphs, making it simpler than using complex RDBMS Joins.
- **Indexing:** Neo4j supports indexing, utilizing tools like Apache Lucene.
- **Indexing Application:** Indexes can be applied to properties of nodes and properties of relationships (edges), allowing traversal to begin by querying the index to find a specific starting node or edge by its property value.

Q9: Provide the advantages and disadvantages/limitations of using Graph Databases, noting concerns such as horizontal scaling introducing poor performance and the difficulty of updating all nodes with a given parameter.

GDBs excel with frequent schema changes and relationship queries, but suffer limitations in horizontal scaling and global updates.

- **Advantage: Schema Flexibility:** Graph models are adept at handling frequent schema changes.
- **Advantage: Real-Time Performance:** GDBs provide fast, real-time query response time, crucial for intelligent data activation and managing large volumes of data.
- **Disadvantage: Horizontal Scaling:** If the application must scale horizontally using sharding, this approach may introduce poor performance due to the relationship-oriented nature of the data.
- **Disadvantage: Global Updates Inefficiency:** GDBs are not very efficient when there is a requirement to update all nodes with a given parameter, as changing a property on all nodes is not a straightforward operation.
- **Disadvantage: Global Graph Operations:** Some graph databases may be unable to handle vast amounts of data, especially when dealing with complex global graph operations.

2 Marks

Q10: What are the four properties summarized by the acronym ACID, which Neo4j supports?

ACID is the standard for transactional reliability supported by Neo4j.

- Definition: Neo4j supports the full set of ACID rules, which are essential for transaction integrity.
- Properties: Atomicity, Consistency, Isolation, and Durability.

Q11: What is the name of the declarative query language used by Neo4j that employs ASCII-art for depicting graphs?

Neo4j's query language simplifies graph traversal.

- Language Name: The powerful declarative query language provided by Neo4j is known as Cypher.
- Syntax Detail: Cypher uses ASCII-art for depicting graphs, which aids in its ease of learning.

Q12: Graph databases are commonly referred to as a sub-type of what broader database category?

GDBs fall under the modern category of non-relational storage.

- Category: Graph databases are commonly referred to as a sub-type of NoSQL databases.

Q13: Name three popular Graph Databases other than Neo4j.

There are several alternatives to the popular Neo4j system.

- Examples: Other popular graph databases include Oracle NoSQL Database, OrientDB, and InfiniteGraph.

Q14: What happens if operations are attempted in Neo4j without wrapping them in transactions?

Transactions are mandatory for write operations in Neo4j.

- Effect: Without wrapping operations that change nodes or add relationships in transactions, an error will occur.
- Error Type: The system will throw a `NotInTransactionException`.

Q15: In a GDB, what structure represents the relationships between nodes?

Connectivity is defined by the graph structure linking entities.

- Structure Name: Edges.
- Function: The edges represent the relationships between the nodes.

Q16: In the context of RDBMS vs. Graph Databases, what GDB operation is used instead of RDBMS 'Joins'?

GDBs use a movement-based operation for relationship navigation.

- Operation Name: Traversal.
- Context: Traversal is the mechanism used in Graph Databases to navigate relationships, replacing the need for RDBMS Joins.